

1.

Explicação: Cria-se um conjunto com oito documentos o qual será usado nos testes do motor de busca. Depois, verifica a quantidade de documentos e consulta o d5

Explorar: Alterar o texto do d5 e consultá-lo novamente

Prever: A quantidade continuará sendo 8, mas o conteúdo de d5 será diferente

```
docs <- c(
  d1 = "recuperacao de informacao ordena documentos por relevancia",
  d2 = "o modelo de espaco vetorial representa documentos como vetores",
  d3 = "bm25 e um modelo probabilistico de ranqueamento de texto",
  d4 = "aprendizado estatistico fundamenta a recuperacao moderna",
  d5 = "o indice invertido acelera a busca em muitos documentos",
  d6 = "embeddings capturam a semantica de palavras e documentos",
  d7 = "a avaliacao mede a relevancia dos resultados da busca",
  d8 = "ciencia de dados combina estatistica e programacao"
)
length(docs)
docs["d5"]
```

8

d5: 'o indice invertido acelera a busca em muitos documentos'

```
docs <- c(
  d1 = "recuperacao de informacao ordena documentos por relevancia",
  d2 = "o modelo de espaco vetorial representa documentos como vetores",
  d3 = "bm25 e um modelo probabilistico de ranqueamento de texto",
  d4 = "aprendizado estatistico fundamenta a recuperacao moderna",
  d5 = "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx",
  d6 = "embeddings capturam a semantica de palavras e documentos",
  d7 = "a avaliacao mede a relevancia dos resultados da busca",
  d8 = "ciencia de dados combina estatistica e programacao"
)
length(docs)
docs["d5"]
```

8

d5: 'xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx'

2.

Explicação: Vai dividir os documentos em palavras menores, chamadas tokens, preparando os textos para as próximas etapas da busca

Explorar: Adicionar ou remover uma palavra de um documento e tokenizá-lo novamente

Prever: Os tokens serão alterados de acordo com a mudança feita no texto

```
tokenizar <- function(texto) {
  texto <- tolower(texto)
  unlist(strsplit(texto, "\\s+"))
}
tokens <- lapply(docs, tokenizar)
tokens[["d1"]]
```

'recuperacao' · 'de' · 'informacao' · 'ordena' · 'documentos' · 'por' · 'relevancia'

3.

Explicação: Ele identifica os termos diferentes do corpus e mostra quais aparecem com maior frequência

Explorar: Repetir uma palavra várias vezes em um documento

Prever: A frequência dessa palavra aumentará e ela poderá subir entre os termos mais frequentes

```
vocab <- sort(unique(unlist(tokens)))
length(vocab)

freq <- table(unlist(tokens))
sort(freq, decreasing = TRUE)[1:6]
```

45

de	a	documentos	e	busca	modelo
6	5	4	3	2	2

4.

Explicação: Vai organizar os termos em uma matriz e mostra quantas vezes cada palavra aparece em cada documento

Explorar: Adicionar uma palavra a um documento e gerar a matriz novamente

Prever: A frequência dessa palavra deverá aumentar no documento alterado

```
tdm <- sapply(tokens, function(tk) {
  as.integer(table(factor(tk, levels = vocab)))
})
```

```
rownames(tdm) <- vocab
tdm[1:6, ]
```

A matrix: 6 × 8 of type int

	d1	d2	d3	d4	d5	d6	d7	d8
a	0	0	0	1	1	1	2	0
acelera	0	0	0	0	1	0	0	0
aprendizado	0	0	0	1	0	0	0	0
avaliacao	0	0	0	0	0	0	1	0
bm25	0	0	1	0	0	0	0	0
busca	0	0	0	0	1	0	1	0

5.

Explicação: Ele procura um termo no corpus e mostra quais documentos possuem essa palavra Ainda não existe uma ordem de relevância entre eles

Explorar: Trocar o termo pesquisado por outra palavra do corpus

Prever: Serão mostrados apenas os documentos que contêm o termo escolhido

```
busca_booleana <- function(termo, tdm) {
  termo <- tolower(termo)
  if (!termo %in% rownames(tdm)) return(character(0))
  colnames(tdm)[tdm[termo, ] > 0]
}
```

6.

Explicar: Ele vai atribuir pesos aos termos considerando sua frequência e sua presença nos documentos, ajudando a identificar palavras mais relevantes

Explorar: Comparar o peso de uma palavra comum com o de uma palavra que aparece em poucos documentos

Prever: A palavra mais rara deverá receber um peso maior do que uma palavra muito comum

```
N <- ncol(tdm)
df <- rowSums(tdm > 0)
idf <- log(N / df)
tfidf <- tdm * idf

round(tfidf[c("documentos", "recuperacao", "busca", "de"), ], 2)
```

A matrix: 4 × 8 of type dbl

	d1	d2	d3	d4	d5	d6	d7	d8
documentos	0.69	0.69	0.00	0.00	0.69	0.69	0.00	0.00
recuperacao	1.39	0.00	0.00	1.39	0.00	0.00	0.00	0.00
busca	0.00	0.00	0.00	0.00	1.39	0.00	1.39	0.00
de	0.47	0.47	0.94	0.00	0.00	0.47	0.00	0.47

7. Explicação: Ele busca o conteúdo de um artigo da Wikipédia para substituir o pequeno corpus por textos reais e maiores

Explorar: Trocar o artigo de Santos pelo de outro município da Baixada Santista

Prever: O programa vai retornar um texto diferente, correspondente ao novo artigo escolhido

```
library(httr2)

baixar_wiki <- function(titulo) {
```

```
request("https://pt.wikipedia.org/w/api.php") |>
req_url_query(action = "query", prop = "extracts", explaintext = 1,
              format = "json", redirects = 1, titles = titulo) |>
req_perform() |> resp_body_json() |>
  (\\(r) r$query$pages[[1]]$extract)()
}

texto <- baixar_wiki("Santos (São Paulo)")
substr(texto, 1, 60)
```

'Santos é um município brasileiro no litoral do estado de São'

8.

Explicação: o grep procura partes do texto, enquanto a busca booleana procura termos já existentes na matriz

Explorar: Trocar "recupera" por "modelo" e comparar os resultados

Prever: As duas buscas deverão encontrar resultados relacionados a "modelo", pois essa palavra existe como um termo completo no corpus

```
grep("recupera", docs)
busca_booleana("recupera", tdm)
```

1 · 4

```
grep("recupera", docs)
busca_booleana("modelo", tdm)
```

1 · 4
'd2' · 'd3'